

AI-Based Project Risk Forecasting System

Combined Knowledge Base — RAG Reference Document

This document combines all six knowledge-base source documents into a single reference file, covering the complete project storyline:

1. Project Overview & Requirements

2. System Architecture & Technologies

3. Dataset & Data Details

4. AI/ML Model Details

5. Testing & Performance Results

6. Project Status & Decision Log

Project Overview & Requirements

AI-Based Project Risk Forecasting System — objectives, scope, and technical requirements

Project: AI-Based Project Risk Forecasting System | Prepared by: Abikrishna, B.E. Computer Science and Engineering | Infosys Springboard AI Internship | Compile Journey

1. Project Background

Software and IT projects frequently overrun their planned budget and schedule. In most organizations, identifying which projects are at risk still depends heavily on the subjective judgement of a project manager, reviewed periodically rather than continuously. This creates a gap: risk is frequently recognized only after it has already begun to affect delivery, at which point options for correction are limited.

The AI-Based Project Risk Forecasting System was built to close that gap. It is a machine learning system that forecasts a project's risk level using only information available at or before the project's kickoff — budget, team composition, complexity, governance maturity, and related planning-stage attributes. The goal is to give project managers an early, data-driven signal so mitigation can begin before problems compound.

2. Problem Statement

Given a set of project planning attributes known before execution begins, predict the project's risk category (Low, Medium, High, or Critical) so that project managers can take proactive mitigation steps rather than reactive ones.

3. Project Objectives

- **Forecast risk before kickoff** — classify a project into Low / Medium / High / Critical risk using only information known before the project starts.
- **Avoid hindsight bias** — deliberately exclude outcome data (actual cost, delays, defect counts) so the model reflects genuine forecasting rather than lookback.
- **Build an explainable model** — use interpretable features such as priority, complexity, and governance load so predictions can be justified to stakeholders.
- **Deliver a usable tool** — wrap the trained model in a web interface where a project manager enters project details and receives an instant risk forecast.

4. Project Context

Field	Detail
Project title	AI-Based Project Risk Forecasting System
Prepared by	Abikrishna, B.E. Computer Science and Engineering

Internship program	Infosys Springboard AI Internship
Personal brand	Compile Journey
Domain	Project management + machine learning (tabular classification)
Prediction target	risk_category (Low / Medium / High / Critical)
Primary dataset	project_risk_dataset.csv (200,000 rows, 33 columns)

5. Scope

In scope: dataset evaluation and selection, data cleaning, feature engineering, model training and evaluation, results documentation, and UI design for the forecasting tool. Out of scope for the current phase: a fully deployed, live production web application (the application layer exists as a validated design/mockup; backend wiring to the trained model is a pending task — see Document 6).

6. Software & Environment Requirements

- Python 3.10 or later
- Google Colab or Jupyter Notebook as the development environment
- No GPU required — model training runs comfortably on CPU

7. Core Libraries

Library	Purpose
pandas, numpy	Data loading, cleaning, and manipulation
scikit-learn	Preprocessing (label encoding), train/test splitting, evaluation metrics
xgboost	Final gradient-boosted classification model
joblib	Persisting the trained model and encoders to disk
matplotlib	Chart generation for reporting and analysis

8. Data Requirements

- project_risk_dataset.csv — 200,000 rows, 33 columns
- Clean CSV, UTF-8 encoded
- No missing values or duplicate handling required prior to training (verified during data cleaning)

9. Planned Deployment Requirements

- Web front-end providing an input form and a result view
- Backend API serving predictions from the saved .joblib model
- Hosting on any standard Python web stack

10. Stakeholders

Role	Party
Project owner / developer	Abikrishna
Reviewing mentor	Infosys Springboard AI Internship mentor
End users (intended)	Project managers assessing new project risk at planning stage

11. Why This Project Matters

Traditional project risk assessment relies on checklists, status meetings, and the accumulated intuition of experienced project managers. This works reasonably well for managers who have run dozens of similar projects, but it does not scale: it is inconsistent across teams, hard to audit, and slow to apply to a new project the moment it is proposed. A machine-learned forecasting model addresses all three weaknesses at once. It applies the same, auditable logic to every project. It can be run the instant a project's planning parameters are known — before a single day of work has started. And because it is trained on a large historical dataset (200,000 projects), it can recognize risk patterns that would take a single human project manager an entire career to observe directly.

The project deliberately treats explainability as a first-class requirement, not an afterthought. Every prediction the model makes can be traced back to the specific features that drove it (see Document 4, Feature Importance), which means the tool can be used to support a human project manager's judgement rather than replace it opaquely.

12. Success Criteria

The project defines success along three dimensions, all of which have been met as of this document set:

- **Accuracy criterion:** the model should meaningfully outperform a naive baseline. Achieved — 80.4% accuracy vs. a 45% majority-class baseline.
- **Integrity criterion:** the model's accuracy must reflect genuine forecasting, not information leaked from project outcomes. Achieved — all outcome/leakage columns were identified and excluded (see Document 3, Section 4).
- **Explainability criterion:** the top features driving predictions must be interpretable by a non-technical project manager. Achieved — the leading features (priority, technical complexity, regulatory compliance load) are all recognizable project-management concepts.

13. Frequently Asked Questions

Q: What exactly does this project do?

It predicts, before a project starts, which risk category (Low, Medium, High, or Critical) the project is likely to fall into, based on planning-stage attributes such as budget, team size, complexity, and governance maturity.

Q: Who is the intended user of this system?

Project managers and PMO (Project Management Office) staff who need an early, consistent risk signal when a new project is proposed or planned, before execution begins.

Q: What makes this different from a project manager's own judgement?

It is consistent (the same inputs always produce the same prediction), auditable (every prediction can be traced to specific contributing features), and trained on a much larger set of historical projects (200,000) than any individual manager could personally have overseen.

Q: Is this project complete?

The data science portion — dataset selection, cleaning, model training, and evaluation — is complete and validated. The web application that would let a user interactively query the model is designed but not yet deployed; see Document 6 for the full status breakdown.

Q: What programming language and tools were used?

Python 3.10+, using pandas and numpy for data handling, scikit-learn for preprocessing and evaluation, xgboost for the model itself, and joblib to save the trained model. Development took place in Google Colab / Jupyter Notebook.

Q: Does this project require a GPU?

No. The final model trains comfortably on CPU in well under a few minutes for the full 200,000-row dataset.

System & Technical Architecture

Architecture layers, technology stack, pipeline, and application design

Project: AI-Based Project Risk Forecasting System | Prepared by: Abikrishna, B.E. Computer Science and Engineering | Infosys Springboard AI Internship | Compile Journey

1. Architecture Overview

The system is organized into three layers. The **data layer** holds the cleaned, 200,000-row dataset restricted to the 19 forecast-time features (see Document 3 for the full feature list). The **model layer** is the trained XGBoost classifier (80.4% held-out accuracy), persisted as a .joblib file along with its label encoders. The **application layer** is a web interface where a project manager enters project details into a form and receives an instant risk forecast, served by a backend that loads the saved model and returns a prediction.

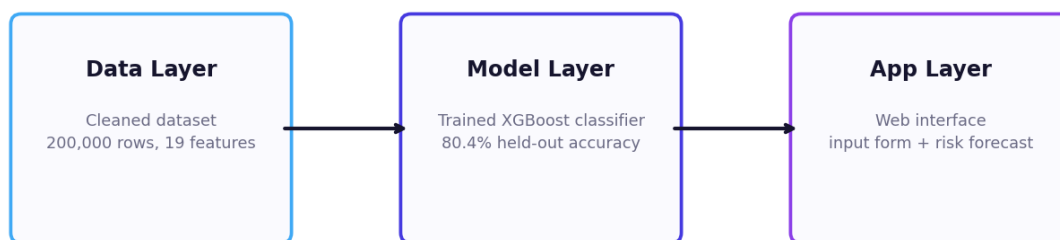


Figure 2.1: Three-layer system architecture.

2. End-to-End Pipeline

- **Step 1 — Load & Inspect:** read the 200,000-row dataset and confirm zero missing values and zero duplicates.
- **Step 2 — Select Features:** retain only pre-project columns; explicitly exclude outcome/leakage columns.
- **Step 3 — Encode:** label-encode categorical columns (project_type, region, methodology, etc.).
- **Step 4 — Split:** stratified 80/20 train/test split so the test set is a genuine held-out sample.
- **Step 5 — Train:** fit an XGBoost classifier on the training set.
- **Step 6 — Evaluate:** score on held-out test data using accuracy, F1, confusion matrix, and feature importance.
- **Step 7 (planned) — Deploy:** serve the trained model behind a backend API consumed by the web front-end.

3. Technology Stack

Layer / Purpose	Technology
Language	Python 3.10+
Development environment	Google Colab / Jupyter Notebook
Data handling	pandas, numpy
Preprocessing & metrics	scikit-learn (LabelEncoder, train_test_split, classification_report, confusion_matrix)
Model	xgboost (XGBClassifier)
Model persistence	joblib
Charting / reporting	matplotlib
Compute	CPU only — no GPU required
Planned front-end	Web form (input) + result view (output)
Planned backend	API serving the saved .joblib model
Planned hosting	Standard Python web stack

4. Final Model Configuration

The deployed model is an XGBoost classifier (XGBClassifier) configured as follows: **n_estimators=500**, **max_depth=7**, **learning_rate=0.05**, **subsample=0.85**, **colsample_bytree=0.85**, **reg_lambda=1.2**, **random_state=42**. Full training/evaluation methodology is documented in Document 4.

5. Application Design

The application is designed around a two-screen flow: an input form where the project manager enters known project attributes, and a result screen showing the forecast.

5.1 Input Screen Fields

- Priority, Project Type, Industry Sector, Methodology
- Planned Duration (days), Team Size, Team Avg. Experience (years)
- Budget (USD), Stakeholder Count, Vendor Dependency Count
- Tech Complexity Score, Regulatory Compliance Load, Scope Clarity Score
- Sponsor Engagement Score, Communication Score, Previous Project Success Rate (%)
- Region, Contract Type, External Dependency Score

5.2 Output Screen Contents

- Predicted Risk Level (Low / Medium / High / Critical) with a visual badge
- Confidence percentage for the predicted class
- Full class probability breakdown across all four risk levels
- Top contributing factors behind the specific prediction

- A recommended action line (e.g. escalate for PM review) for higher-risk predictions

6. Deployment Status

As of this document, the data layer and model layer are fully built and validated. The application layer exists as a validated UI design; wiring the front-end to a live backend serving the trained model is a pending task (see Document 6 for full status).

7. Why a Three-Layer Architecture

The system is deliberately separated into three independent layers rather than built as a single monolithic script, for three practical reasons. First, **separation of concerns**: the data layer can be re-validated or refreshed with a new dataset without touching the model, and the model can be retrained without touching the application layer. Second, **testability**: each layer can be tested in isolation — the model layer's accuracy is measured against a held-out test set independent of any UI concerns. Third, **deployability**: the model layer is packaged as a portable .joblib artifact, meaning the trained model can be moved to a different serving environment without retraining.

8. Data Flow in Detail

- A project manager (or, during development, a script) supplies the 19 pre-project feature values.
- The application layer collects these values from the input form.
- The backend applies the same LabelEncoder transformations used during training to convert categorical fields (project_type, industry_sector, methodology, region, contract_type, priority) into numeric codes.
- The encoded feature vector is passed to the loaded XGBoost model's .predict() and .predict_proba() methods.
- The backend returns the predicted class, a confidence percentage, and the full probability breakdown across all four risk classes.
- The application layer renders this as the result screen: a risk-level badge, a confidence score, a probability bar chart, and a list of top contributing factors.

9. Non-Functional Considerations

Performance

Prediction latency for a single project is expected to be well under 100 milliseconds once the model is loaded in memory, since XGBoost inference on a single 19-feature row is computationally trivial compared to the 500-tree ensemble's training cost.

Portability

Because the model is serialized with joblib, it can be loaded into any Python environment with a matching xgboost version, without needing to re-run training.

Maintainability

The label encoders are saved alongside the model (feature_encoders.joblib, label_encoder.joblib) so that new input values are transformed identically to how the training data was transformed, preventing silent encoding mismatches in production.

10. Frequently Asked Questions

Q: What model format is the trained model saved in?

A .joblib file (final_risk_model.joblib), which is the standard serialization format used by scikit-learn-compatible Python models, including XGBoost's scikit-learn API.

Q: Does the architecture require a database?

Not for the core prediction functionality. The model itself is stateless; a database would only be needed if the deployed application later adds features like storing prediction history.

Q: Can the model be retrained with new data later?

Yes. Because feature selection, encoding, and training are all scripted (not manual), the same pipeline can be re-run against an updated or expanded dataset to produce a refreshed model file.

Q: What is the single biggest architectural gap right now?

The backend API layer that would connect the front-end form to the trained model has not yet been built — this is the primary pending task described in Document 6.

Dataset & Data Details

Data sources, dataset evaluation history, features, preprocessing, and target definition

Project: AI-Based Project Risk Forecasting System | Prepared by: Abikrishna, B.E. Computer Science and Engineering | Infosys Springboard AI Internship | Compile Journey

1. Final Dataset Overview

The project uses a 200,000-row project-risk dataset (project_risk_dataset.csv) covering 33 columns spanning project fundamentals, team and delivery history, process and governance, contextual/complexity factors, and post-execution outcome measures. The dataset is clean by construction: it contains zero missing values and zero duplicate rows or project identifiers across the full 200,000 records, and required no imputation or synthetic augmentation before use.

Metric	Value
Total records	200,000
Total columns	33
Forecast-time features used	19
Risk classes	4 (Low, Medium, High, Critical)
Missing values	0
Duplicate rows / IDs	0

2. Target Variable

The prediction target is **risk_category**, a four-level classification derived from a continuous 0–100 **risk_score** using fixed thresholds: Low = 0–30, Medium = 30–55, High = 55–75, Critical = 75–100.

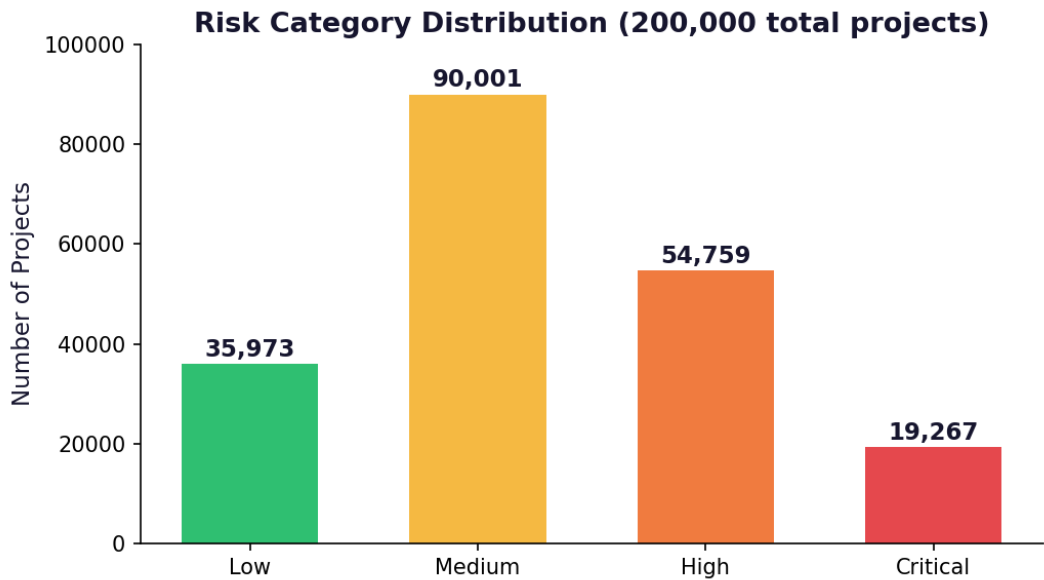


Figure 3.1: Risk category distribution across all 200,000 projects.

Risk Category	Count	Share
Medium	90,001	45.0%
High	54,759	27.4%
Low	35,973	18.0%
Critical	19,267	9.6%

3. Feature Categories

Project Fundamentals

priority, planned_duration_days, budget_usd, team_size, stakeholder_count

Team & Delivery History

team_avg_experience_years, previous_project_success_rate_pct, vendor_dependency_count

Process & Governance

methodology, regulatory_compliance_load, scope_clarity_score, sponsor_engagement_score

Context & Complexity

project_type, industry_sector, region, contract_type, tech_complexity_score, external_dependency_score

Outcome Fields (present in raw data, excluded from the model)

actual_cost_usd, cost_overrun_pct, defect_count, milestones_missed, risk_score, project_status

4. Data Leakage — Identification and Handling

A critical step in this project was recognizing that the raw dataset contains two distinct kinds of columns: attributes known before a project starts, and attributes only known once the project is under way or complete. Training on the full column set — including outcome fields — produced a misleadingly high accuracy of **99.7%**. This is not a genuine forecasting result; it reflects the model reading outcomes rather than predicting risk in advance. All such columns were identified and removed before training the final model.

USED — known before project starts	EXCLUDED — known only during/after
priority, planned_duration_days, team_size, team_avg_experience_years, stakeholder_count, budget_usd, tech_complexity_score, regulatory_compliance_load, scope_clarity_score, sponsor_engagement_score, communication_score, previous_project_success_rate_pct, project_type, industry_sector, region, contract_type, methodology (19 columns total)	actual_duration_days, actual_cost_usd, cost_overrun_pct, schedule_overrun_pct, resource_availability_pct, team_turnover_pct, requirement_changes_count, safety_incidents, defect_count, milestones_missed, risk_score (the label source), project_status (11 columns total)

5. Final Feature Set (19 features)

project_type, industry_sector, methodology, region, contract_type, priority, planned_duration_days, team_size, team_avg_experience_years, stakeholder_count, budget_usd, vendor_dependency_count, communication_score, sponsor_engagement_score, previous_project_success_rate_pct, tech_complexity_score, regulatory_compliance_load, scope_clarity_score, external_dependency_score.

A follow-up ablation test confirmed the **region** column contributes very little predictive value on its own: removing it changed held-out accuracy from 80.45% to 80.36%, a difference of only 0.09 percentage points. It was retained since it does no harm, but the model's predictive strength does not depend on it.

6. Alternative Datasets Evaluated and Rejected

Three other dataset candidates were tested before this one was confirmed as the best fit. Genuine accuracy figures below all use the same leakage-free evaluation standard (pre-project features only, held-out test set).

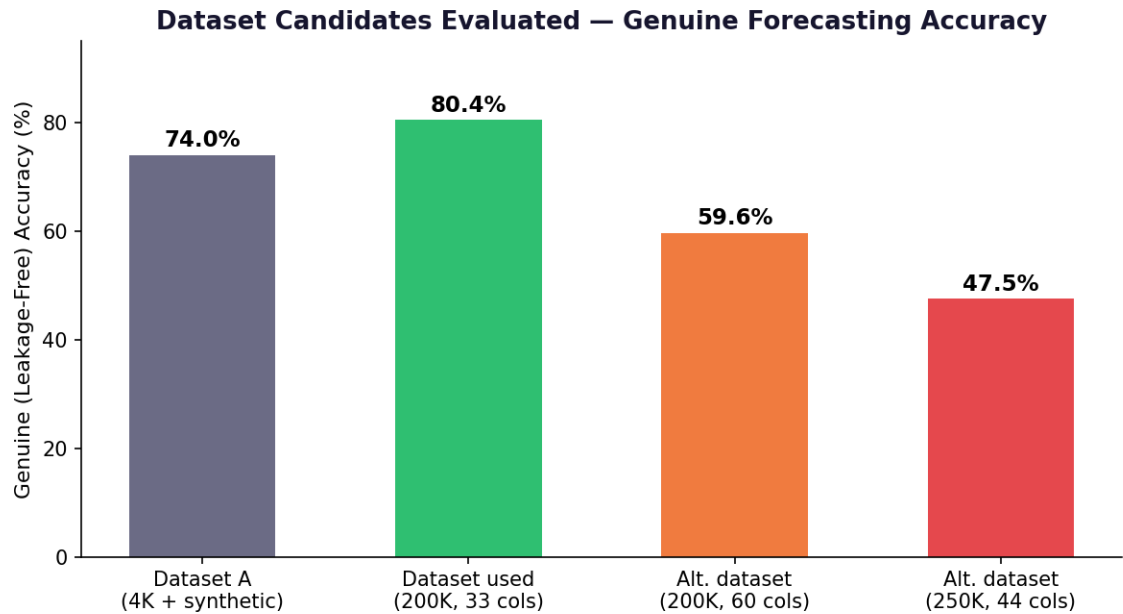


Figure 3.2: Genuine forecasting accuracy across all dataset candidates evaluated.

Dataset	Size	Genuine Accuracy	Outcome
Initial dataset	4,000 real rows (+6,500 synthetic)	55–74% (varied by framing)	Rejected — too small, needed synthetic augmentation
project_risk_dataset.csv (used)	200,000 rows, 33 cols	80.4%	Selected — best genuine accuracy
Alt. 60-column dataset	200,000 rows, 60 cols	59.6%	Rejected — too many outcome-tracking columns diluted signal
risk_forecasting_dataset.csv	250,000 rows, 44 cols	47.5%	Rejected — near-random; Critical recall only 3.7%

7. Data Cleaning Steps Applied

- Verified zero missing values across all columns (no imputation needed on the final dataset)
- Verified zero duplicate rows and zero duplicate project IDs
- Label-encoded all categorical columns (project_type, industry_sector, methodology, region, contract_type, priority) for model training
- Confirmed risk_category class balance was sufficient across all four classes to avoid a majority-class-only model

8. Data Limitation Note

This dataset is a large, clean, globally-representative dataset rather than one sourced specifically from real Indian organizations. The closest available regional grouping ("Asia Pacific") accounts for 22.1% of rows and cannot be isolated to India specifically. This is disclosed transparently rather than presented as India-specific data.

9. Column-Level Reference — Features Used

Column	Meaning
project_type	Category of the project (e.g., Software Development, Construction, ERP Implementation)
industry_sector	Industry the project is being delivered for (e.g., Finance, Healthcare, IT)
methodology	Delivery methodology (Agile, Waterfall, or Hybrid)
region	Geographic region of delivery (e.g., North America, Europe, Asia Pacific)
contract_type	Commercial contract structure (Fixed Price, Time & Material, Cost Plus)
priority	Organizational priority level assigned to the project (Low/Medium/High/Critical)
planned_duration_days	Planned project duration in days, set at kickoff
team_size	Number of people planned on the project team
team_avg_experience_years	Average years of experience across the planned team
stakeholder_count	Number of stakeholders involved in the project
budget_usd	Planned project budget in US dollars
vendor_dependency_count	Number of external vendors the project depends on
communication_score	Planned communication cadence/quality score
sponsor_engagement_score	Expected level of executive sponsor engagement
previous_project_success_rate_pct	Historical success rate (%) of similar past projects
tech_complexity_score	Technical complexity rating assigned at planning stage
regulatory_compliance_load	Degree of regulatory/compliance burden on the project
scope_clarity_score	How clearly the project scope is defined at kickoff
external_dependency_score	Degree of dependency on factors outside the project team's control

10. Column-Level Reference — Excluded Outcome Columns

Column	Why it is excluded
actual_duration_days	Only known once the project has finished — an outcome, not a plan
actual_cost_usd	Actual spend, only known during/after execution
cost_overrun_pct	Directly derived from actual vs. planned cost — an outcome metric
schedule_overrun_pct	Directly derived from actual vs. planned schedule — an outcome metric
resource_availability_pct	Reflects real-time resourcing during execution, not planning-stage information
team_turnover_pct	Accumulates as the project runs; unknown at kickoff

requirement_changes_count	Accumulates during execution as requirements evolve
safety_incidents	An outcome that can only occur once work has started
defect_count	A quality metric only measurable once development/testing has occurred
milestones_missed	A delivery outcome, only known partway through or after the project
risk_score	The continuous score risk_category is directly derived from — using it would be circular
project_status	Completed / Delayed / Failed — by definition only known at or after project end

11. Glossary of Key Data Terms

Term	Definition
Data leakage	When a model is trained using information that would not actually be available at prediction time, producing artificially inflated accuracy
Held-out test set	A portion of data set aside and never used during training, used only to measure genuine performance
Stratified split	A train/test split that preserves the same class proportions in both the training and test sets
Label encoding	Converting categorical text values (e.g., "Agile", "Waterfall") into numeric codes a model can process
Feature importance	A measure of how much each input feature contributed to the model's predictions
Ablation test	An experiment that removes one feature (or component) at a time to measure its individual contribution

12. Frequently Asked Questions

Q: How many rows and columns does the final dataset have?

200,000 rows and 33 columns in the raw file; 19 of those columns are used as model features after removing identifiers, the target, and outcome/leakage columns.

Q: Were there any missing values to handle?

No. The final dataset (project_risk_dataset.csv) has zero missing values and zero duplicate rows across all 200,000 records, so no imputation was required.

Q: What is risk_score and how does it relate to risk_category?

risk_score is a continuous value from 0 to 100. risk_category is a 4-level bucket derived from it using fixed thresholds: Low 0–30, Medium 30–55, High 55–75, Critical 75–100. risk_score itself is excluded from the model's features because it is the direct source of the label.

Q: Why wasn't the dataset augmented with synthetic data?

Because it is already large (200,000 rows) and clean (no missing values, no duplicates), unlike the very first 4,000-row dataset used earlier in the project, which did require synthetic augmentation to reach a workable training size.

Q: How was data leakage discovered?

By training an initial model on all 33 raw columns and observing a suspiciously high 99.7% accuracy. Investigating which columns were driving that accuracy revealed that several were outcome measurements (e.g., actual cost, defect count) rather than planning-stage information.

Q: Is the region column necessary?

Not really — an ablation test showed removing it only changes accuracy from 80.45% to 80.36%, a 0.09 percentage point difference. It was kept because it does no harm, not because it is essential.

AI / ML Model Details

Algorithms evaluated, training process, final configuration, and evaluation metrics

Project: AI-Based Project Risk Forecasting System | Prepared by: Abikrishna, B.E. Computer Science and Engineering | Infosys Springboard AI Internship | Compile Journey

1. Modeling Task

This is a **multi-class classification** problem: given 19 pre-project features, predict which of four risk categories (Low, Medium, High, Critical) a project belongs to. The model is trained and evaluated exclusively on features known before a project starts (see Document 3 for the full feature list and the data leakage analysis that shaped this constraint).

2. Model Selection

Three model families were compared under identical train/test conditions on held-out real data. XGBoost, a gradient-boosted tree ensemble, produced the strongest result and was selected as the final model.

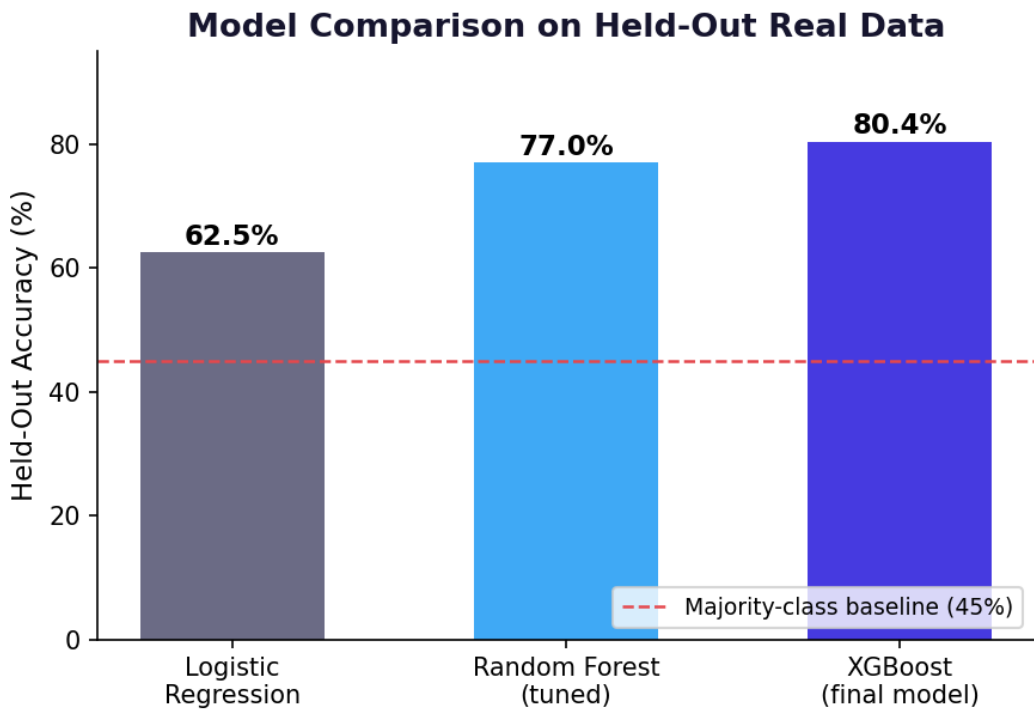


Figure 4.1: Held-out accuracy by model family, against the 45% majority-class baseline.

Model	Accuracy (held-out)	Notes
XGBoost (final model)	80.4%	Best result — handles this tabular feature mix well
Random Forest (tuned)	~76–78%	Solid, but consistently trails XGBoost

Logistic Regression	~60–65%	Too simple for the non-linear feature relationships
---------------------	---------	---

3. Final Model Configuration

Hyperparameter	Value
n_estimators	500
max_depth	7
learning_rate	0.05
subsample	0.85
colsample_bytree	0.85
reg_lambda	1.2
random_state	42
eval_metric	mlogloss

4. Training Process

- **Data split:** stratified 80/20 train/test split (160,000 training rows, 40,000 held-out test rows), preserving class proportions so the rare Critical class is represented in both sets.
- **Encoding:** categorical columns (project_type, industry_sector, methodology, region, contract_type, priority) label-encoded to numeric codes.
- **Fitting:** XGBClassifier trained on the 160,000-row training portion only.
- **Held-out evaluation:** predictions generated on the 40,000-row test set, which the model never saw during training.

5. Code Walkthrough

Code element	Explanation
train_test_split(..., stratify=y_enc)	Creates an 80/20 split that preserves class proportions, so rare classes (Critical) are not lost from either set.
LabelEncoder()	Converts text columns (project_type, region, methodology, etc.) into numbers XGBoost can train on.
XGBClassifier(n_estimators=500, max_depth=7, ...)	Gradient-boosted tree ensemble — builds 500 trees sequentially, each correcting the previous one's errors.
.fit(X_train, y_train)	Trains the model only on the 160,000-row training portion.
.predict(X_test)	Generates predictions on the 40,000-row held-out test set.
classification_report() / confusion_matrix()	Produces per-class precision, recall, F1, and the full error breakdown.

6. Evaluation Metrics — Overall

Metric	Value
Accuracy (held-out)	80.4%
Macro F1	0.790
Weighted F1	0.804
Test set size	40,000 rows
Majority-class baseline	45%

7. Evaluation Metrics — Per Class

Class	Precision	Recall	F1-score
Critical	0.839	0.676	0.748
High	0.761	0.766	0.763
Low	0.850	0.773	0.810
Medium	0.808	0.868	0.837

Recall for the Critical class (67.6%) is the model's weakest point — roughly a third of genuinely Critical projects are predicted as a lower risk level. This is flagged as the top priority for future improvement, since under-predicting Critical risk is the costliest kind of error for a real project manager.

8. Feature Importance

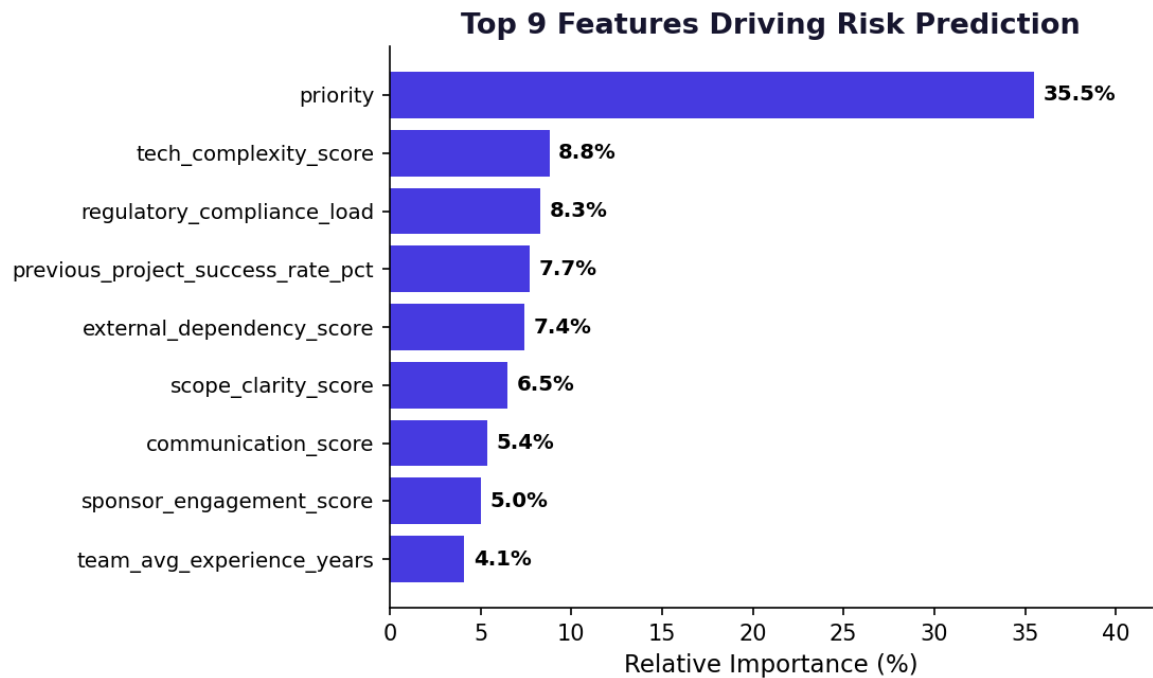


Figure 4.2: Relative importance of the top 9 predictive features.

priority is by far the strongest predictor (35.5% relative importance), followed by `tech_complexity_score` (8.8%), `regulatory_compliance_load` (8.3%), `previous_project_success_rate_pct` (7.7%), and `external_dependency_score` (7.4%). All top features are genuine planning-stage signals that a project manager would reasonably associate with risk, supporting the model's explainability.

9. How XGBoost Works (Plain-Language Explanation)

XGBoost ("Extreme Gradient Boosting") builds a large number of small decision trees — 500, in this model — one after another. The first tree makes rough predictions. Each following tree is trained specifically to correct the mistakes the previous trees made, gradually refining the overall prediction. The final prediction combines the outputs of all 500 trees. This sequential, error-correcting approach is why gradient boosting models like XGBoost tend to outperform a single decision tree, or even a collection of independent trees like Random Forest, on structured/tabular data such as this project's 19-feature dataset.

10. What Each Evaluation Metric Means

Metric	Plain-language meaning
Accuracy	The percentage of all test projects the model classified correctly, across all four risk levels
Precision (per class)	Of all the projects the model predicted as this class, what percentage actually were that class
Recall (per class)	Of all the projects that actually are this class, what percentage the model correctly identified
F1-score	A balance between precision and recall for a given class, useful when both false positives and false negatives matter
Macro F1	The average F1-score across all four classes, treating each class equally regardless of how common it is
Weighted F1	The average F1-score across all four classes, weighted by how many projects are actually in each class
Confusion matrix	A table showing exactly which classes were predicted for each actual class, revealing where errors happen

11. Why Recall Matters More Than Precision for the Critical Class

In a risk-forecasting tool, missing a genuinely Critical project (a false negative, captured by recall) is more costly than incorrectly flagging a safe project as Critical (a false positive, captured by precision). A missed Critical project receives no early mitigation and may fail outright; a false-alarm project simply receives closer-than-necessary attention. This is why Critical-class recall (currently 67.6%) is tracked as the single most important number to improve going forward, even though Critical precision (83.9%) is already strong.

13. Full Training Code

The exact code used to train and evaluate the final model, reproducible in Google Colab or a local Jupyter environment:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
from xgboost import XGBClassifier
import joblib

df = pd.read_csv("project_risk_dataset.csv")
target = "risk_category"

FEATURES = [
    "project_type", "industry_sector", "methodology", "region", "contract_type", "priority",
    "planned_duration_days", "team_size", "team_avg_experience_years", "stakeholder_count",
    "budget_usd", "vendor_dependency_count", "communication_score", "sponsor_engagement_score",
    "previous_project_success_rate_pct", "tech_complexity_score", "regulatory_compliance_load",
    "scope_clarity_score", "external_dependency_score",
]

X = df[FEATURES].copy()
y = df[target].copy()

cat_cols = X.select_dtypes(include="object").columns.tolist()
encoders = {}
for c in cat_cols:
    le = LabelEncoder()
    X[c] = le.fit_transform(X[c].astype(str))
    encoders[c] = le

y_le = LabelEncoder()
y_enc = y_le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc, test_size=0.2, stratify=y_enc, random_state=42
)

final_model = XGBClassifier(
    n_estimators=500, max_depth=7, learning_rate=0.05,
    subsample=0.85, colsample_bytree=0.85, reg_lambda=1.2,
    random_state=42, eval_metric="mlogloss",
)
final_model.fit(X_train, y_train)
pred = final_model.predict(X_test)

acc = accuracy_score(y_test, pred)
f1_macro = f1_score(y_test, pred, average="macro")
print(f"Accuracy: {acc:.4f}    Macro F1: {f1_macro:.4f}")
print(classification_report(y_test, pred, target_names=y_le.classes_))

joblib.dump(final_model, "final_risk_model.joblib")
joblib.dump(encoders, "feature_encoders.joblib")
joblib.dump(y_le, "label_encoder.joblib")
```

14. Data Visualization — Model Inputs

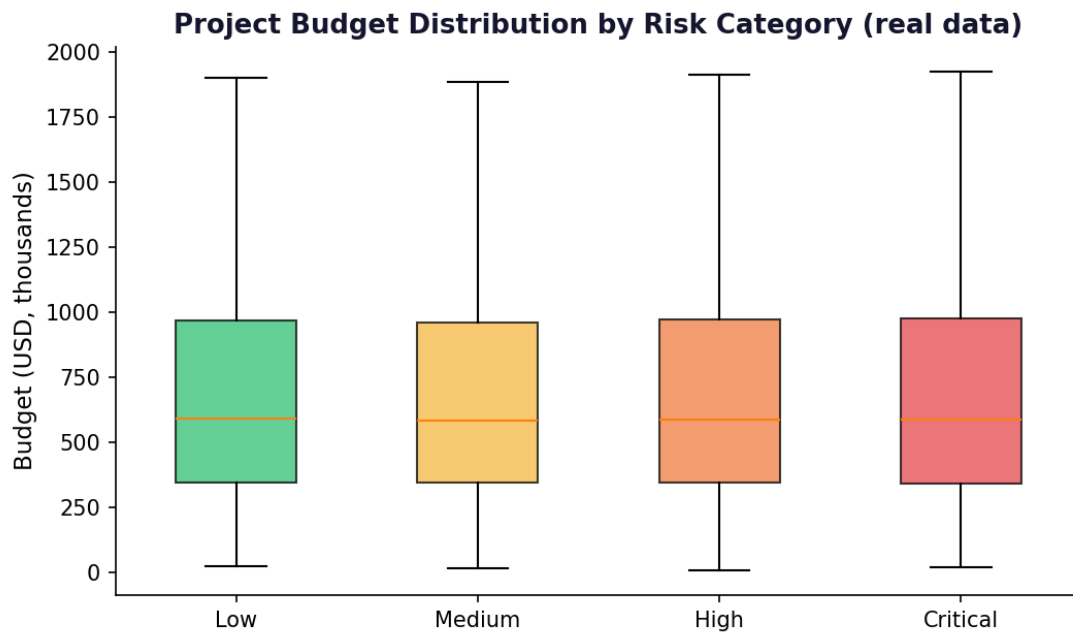


Figure 4.3: Budget distribution by risk category (real data). Median budget is broadly similar across classes, showing risk is not simply a function of project size.

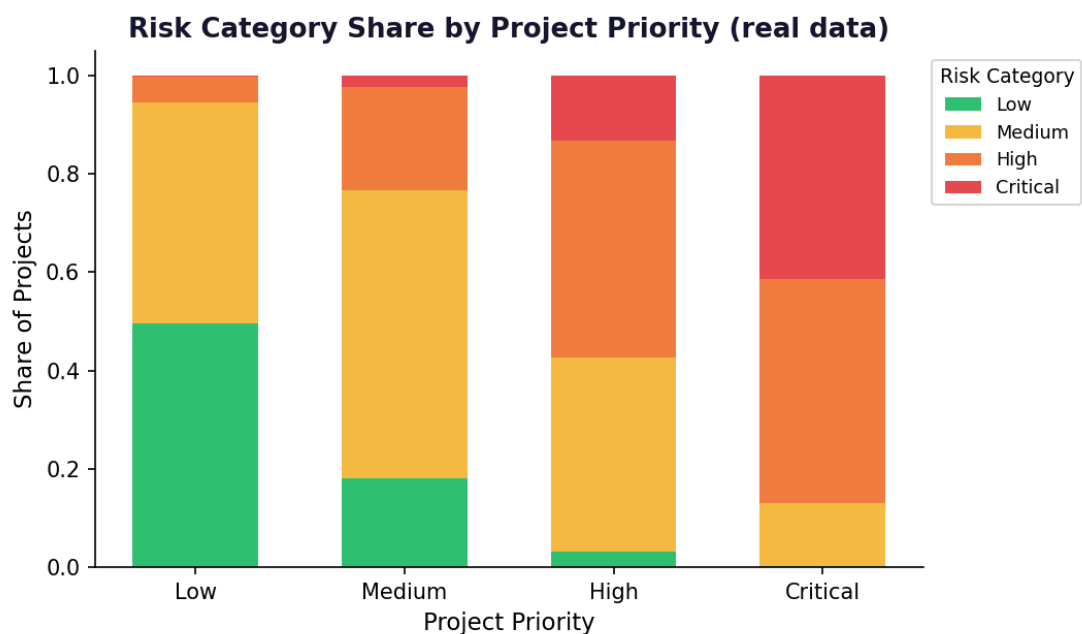


Figure 4.4: Risk category share by project priority (real data), illustrating why priority is the model's strongest predictor.

15. Frequently Asked Questions

Q: What algorithm powers the final model?

XGBoost (Extreme Gradient Boosting), specifically the XGBClassifier from the xgboost Python library.

Q: What is the model's overall accuracy?

80.4% on a held-out test set of 40,000 real projects the model never saw during training.

Q: How does 80.4% compare to random guessing?

Random guessing across 4 classes would score about 25%. Always predicting the most common class (Medium) would score about 45%. The model's 80.4% is well above both baselines.

Q: Which risk class does the model predict least reliably?

Critical, with 67.6% recall — the model misses roughly one in three genuinely Critical projects, classifying them as a lower risk level instead. This is the model's main known weakness.

Q: What is the single most important feature in the model?

priority, which alone accounts for 35.5% of the model's total feature importance — more than double the next most important feature.

Q: Was hyperparameter tuning performed?

Yes. The final configuration (500 trees, max depth 7, learning rate 0.05, 0.85 subsample and column sampling ratios, L2 regularization of 1.2) was selected after comparison against simpler configurations and against Random Forest and Logistic Regression alternatives.

Q: How many features does the model use?

19 features, all known before a project starts. See Document 3 for the complete feature list and the rationale for excluding outcome-based columns.

Testing & Performance Results

Confusion matrix analysis, error patterns, known issues, and dataset testing history

Project: AI-Based Project Risk Forecasting System | Prepared by: Abikrishna, B.E. Computer Science and Engineering | Infosys Springboard AI Internship | Compile Journey

1. Test Methodology

The final model was evaluated on 40,000 held-out real rows that were never used during training (a genuine 20% test split, stratified by risk class). This section documents the detailed error analysis and the issues discovered and resolved during development.

2. Confusion Matrix

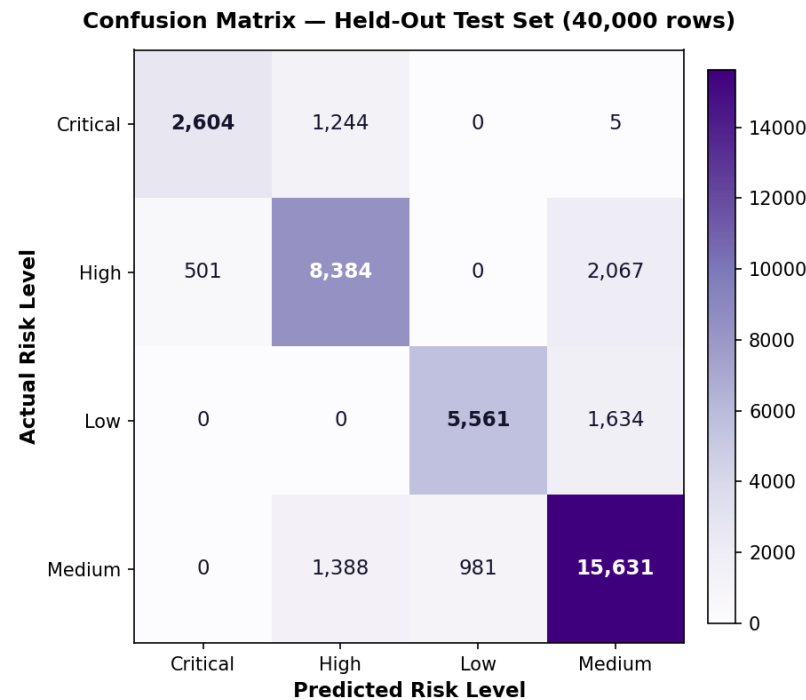


Figure 5.1: Confusion matrix on the held-out test set (rows = actual, columns = predicted).

Actual \ Predicted	Critical	High	Low	Medium
Critical	2,604	1,244	0	5
High	501	8,384	0	2,067
Low	0	0	5,561	1,634
Medium	0	1,388	981	15,631

Error Pattern Analysis

- Almost all misclassifications occur between **adjacent** risk levels (Critical↔High, High↔Medium, Low↔Medium).
- The model **never** confused opposite ends of the risk spectrum — zero Critical projects were predicted as Low, and zero Low projects were predicted as Critical.
- This is the error pattern a risk-forecasting tool should have: when wrong, it is cautious in the right direction rather than randomly wrong.
- The largest single error source is High projects predicted as Medium (2,067 cases) — an under-estimation pattern worth monitoring in production.

3. Known Issue #1 — Data Leakage (Resolved)

Issue: training with outcome/tracking columns included (actual_cost_usd, cost_overrun_pct, schedule_overrun_pct, defect_count, milestones_missed, risk_score, project_status, etc.) produced a misleading 99.7% accuracy. Root cause: the model was reading project outcomes rather than forecasting risk in advance. Resolution: all 11 outcome columns were identified and excluded from the feature set. The corrected, genuine accuracy is 80.4%.

4. Known Issue #2 — Critical-Class Recall Gap (Open)

Issue: recall on the Critical risk class is 67.6%, the weakest of the four classes — meaning approximately one in three genuinely Critical projects is currently predicted as a lower risk level. Status: open, flagged as the top priority for future model improvement (see Document 6). Potential mitigations under consideration: class-weighting, probability threshold tuning, and additional feature engineering around governance and compliance signals.

5. Dataset Candidate Testing History

Multiple dataset candidates were tested end-to-end (clean, feature-select, train, evaluate) before confirming project_risk_dataset.csv as the final choice. Each was scored using the same leakage-free standard for a fair comparison.

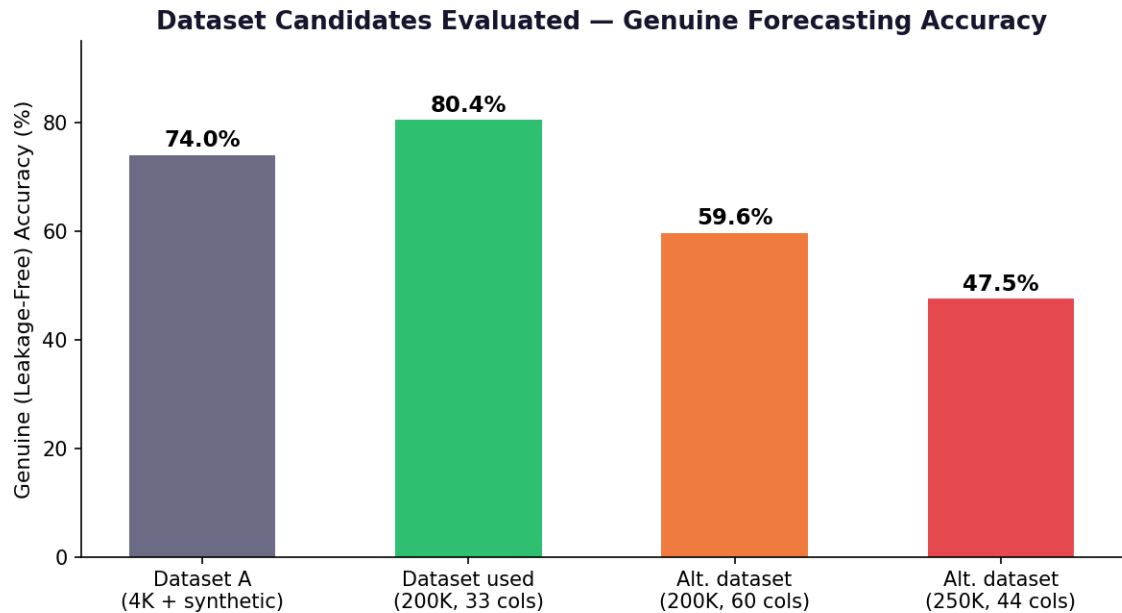


Figure 5.2: Genuine forecasting accuracy across all dataset candidates tested.

Dataset Tested	Result	Decision
Initial 4,000-row dataset + synthetic augmentation	55–74% depending on class framing (4-class vs. 3-class)	Superseded once a larger real dataset was found
project_risk_dataset.csv (200K, 33 cols)	80.4% genuine accuracy	Selected — final dataset in use
Alternative 60-column dataset (200K rows)	59.6% genuine accuracy	Rejected — more outcome-tracking columns diluted usable signal
risk_forecasting_dataset.csv (250K, 44 cols)	47.5% genuine accuracy, 3.7% Critical recall	Rejected — near-random performance, unusable for forecasting

6. Model Comparison Testing

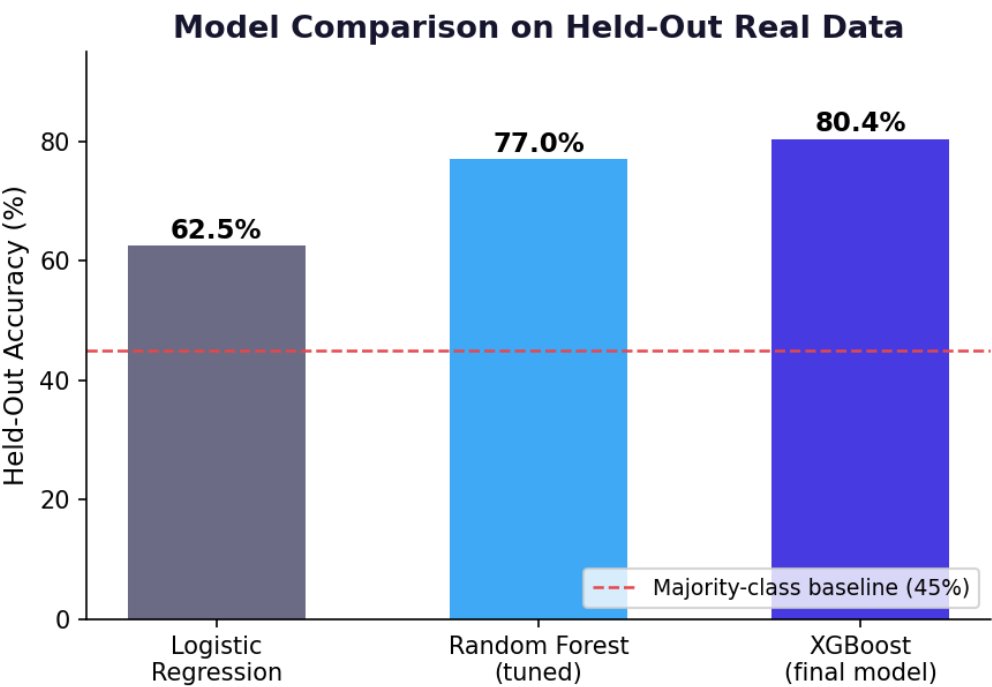


Figure 5.3: Accuracy comparison across all model families tested on the same data.

XGBoost outperformed both Random Forest (tuned) and Logistic Regression by a meaningful margin on identical train/test splits, confirming it as the correct model family for this feature set.

7. Feature Ablation Test

Test	Accuracy	Result
With region feature included (19 features)	80.45%	Baseline
Without region feature (18 features)	80.36%	–0.09 percentage points

Conclusion: region contributes negligible predictive value. It was retained in the final feature set since removing it does not meaningfully change performance.

8. Summary of Testing Outcomes

- Final model accuracy on held-out real data: 80.4% (vs. 45% majority-class baseline)
- Best-performing algorithm confirmed: XGBoost
- Best-performing dataset confirmed: project_risk_dataset.csv (200K rows, 33 columns)
- One critical issue found and resolved during testing: data leakage inflating accuracy to a false 99.7%
- One known issue remains open: Critical-class recall (67.6%), targeted for future improvement

9. Leakage Test Code

The exact comparison used to confirm the leakage issue and measure its impact:

```
# WITH leakage (invalid) vs WITHOUT leakage (genuine)
```

```
leaky_features = [
    "actual_duration_days", "actual_cost_usd", "cost_overrun_pct",
    "schedule_overrun_pct", "resource_availability_pct", "team_turnover_pct",
    "requirement_changes_count", "safety_incidents", "defect_count",
    "milestones_missed", "risk_score", "project_status",
]

# Model A: pre-project features only (genuine forecasting)
acc_genuine = run(FEATURES) # -> 0.8045

# Model B: pre-project features + leaky outcome features (invalid)
acc_leaky = run(FEATURES + leaky_features) # -> 0.9970

print(f"Genuine accuracy: {acc_genuine:.4f}")
print(f"Leaky accuracy: {acc_leaky:.4f} <-- inflated, not real forecasting")
```

10. Data Visualization — Real Dataset Characteristics

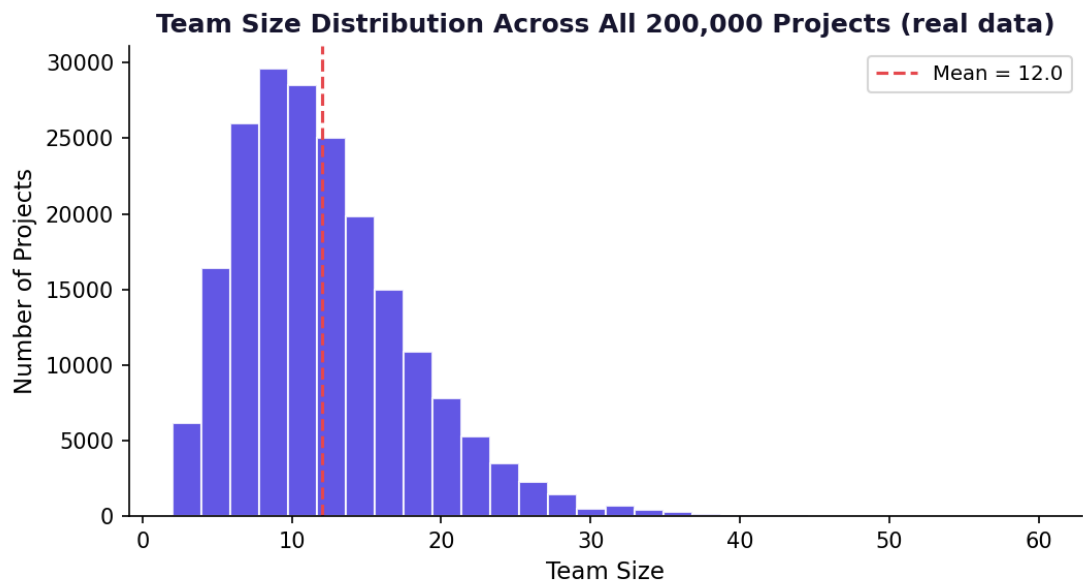


Figure 5.4: Team size distribution across all 200,000 projects (real data), showing a roughly normal spread used as one of the model's 19 input features.

11. Testing Environment

Aspect	Detail
Test set size	40,000 rows (20% stratified held-out split)
Training set size	160,000 rows
Random seed	42 (fixed for reproducibility)
Evaluation library	scikit-learn (classification_report, confusion_matrix, accuracy_score, f1_score)
Hardware	CPU only — no GPU used or required

12. Frequently Asked Questions

Q: What was the biggest problem found during testing?

Data leakage — an initial version of the model, trained with outcome columns included, scored a misleading 99.7% accuracy. This was traced to specific columns and corrected.

Q: What is the model's weakest area today?

Recall on the Critical risk class, currently 67.6%. This is an open issue, not yet resolved, and is the top priority for future model improvement.

Q: Does the model ever confuse Low risk with Critical risk?

No. The confusion matrix shows zero cases of Critical predicted as Low or Low predicted as Critical. All errors occur between adjacent risk levels.

Q: How many dataset candidates were tested in total?

Four: the original small 4,000-row dataset, the final 200,000-row dataset that was selected, and two other 200K+ row alternatives that were tested and rejected for lower genuine accuracy.

Q: What was the worst-performing dataset candidate?

risk_forecasting_dataset.csv (250,000 rows, 44 columns), which achieved only 47.5% genuine accuracy and just 3.7% recall on the Critical class — close to unusable for forecasting purposes.

Project Status & Decision Log

Progress, completed and pending tasks, risks, dependencies, and key decisions

Project: AI-Based Project Risk Forecasting System | Prepared by: Abikrishna, B.E. Computer Science and Engineering | Infosys Springboard AI Internship | Compile Journey

1. Current Project Status

The project has completed dataset selection and validation, data leakage identification and correction, final model training and evaluation, and full documentation (presentation and written report). The remaining major task is building and deploying the live web application around the already-trained model.

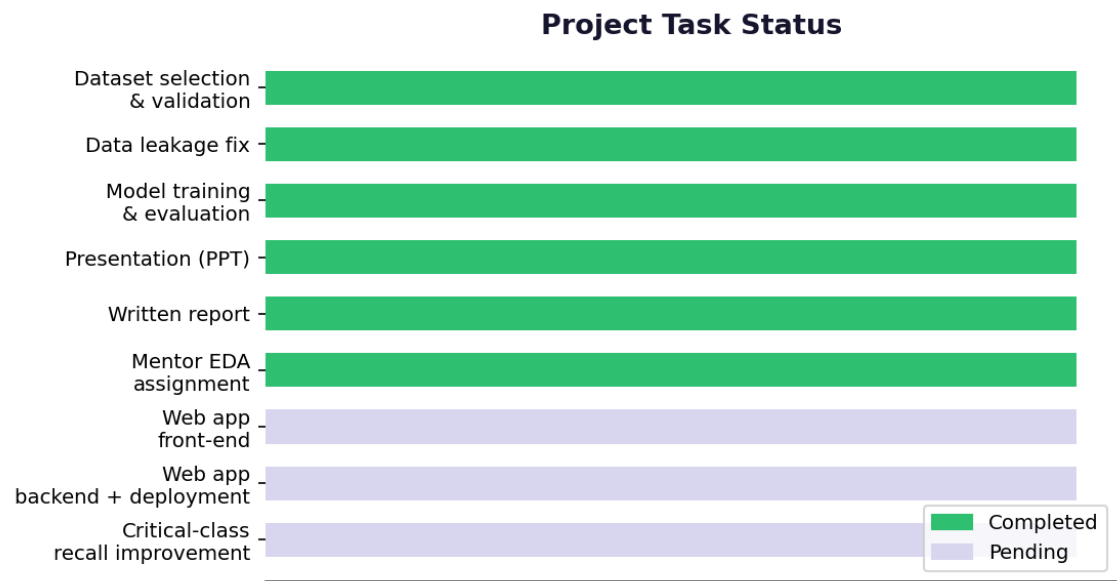


Figure 6.1: Task completion status across the project.

2. Completed Tasks

- Evaluated four candidate datasets (an initial 4,000-row dataset plus three 200K+ row alternatives) and selected the best-performing one (project_risk_dataset.csv, 80.4% genuine accuracy)
- Identified and corrected a data leakage issue that had inflated accuracy to a misleading 99.7%
- Finalized a 19-feature, leakage-free feature set for training
- Compared three model families (Logistic Regression, Random Forest, XGBoost) and selected XGBoost as the final model
- Trained and evaluated the final model: 80.4% held-out accuracy, 0.790 macro F1, full confusion matrix and feature importance analysis completed
- Ran a feature ablation test on the region column, confirming it is safe to keep or drop (± 0.09 pp impact)
- Designed the application UI (input form and result screen) as a validated mockup

- Produced a 21-slide project presentation covering the full storyline
- Produced a 16-page detailed written project report with cover page, table of contents, and figures
- Completed a separate mentor-assigned EDA, data cleaning, and Risk Flag exercise (Phases 1–6) on a dataset copy, including documented column-substitution logic where exact assignment columns were unavailable

3. Pending Tasks

- **Web application front-end:** convert the validated UI mockup into a working, interactive form
- **Backend API:** build a service that loads the saved .joblib model and encoders and returns live predictions
- **End-to-end wiring:** connect front-end to backend so the application produces real, not mocked, predictions
- **Critical-class recall improvement:** investigate class-weighting, threshold tuning, or additional feature engineering to raise Critical recall above its current 67.6%
- **Real-world validation:** if possible, compare model predictions against real project outcomes over time
- **Hosting/deployment:** deploy the application on a standard Python web stack

4. Risks

Risk	Description	Severity
Critical-class recall gap	67.6% recall on the Critical class means roughly 1 in 3 genuinely critical projects may be under-flagged. This is the highest-impact model risk since it directly affects the tool's core purpose.	High
Non-region-specific data	The dataset is a large, globally-representative synthetic-style dataset, not sourced specifically from real Indian organizations. Must be disclosed transparently if region-specific claims are expected.	Medium
Deployment gap	The trained model is not yet wired into a live, usable application — current deliverables are analysis, documentation, and a UI design/mockup rather than a deployed tool.	Medium
Adjacent-class confusion	Errors concentrate between adjacent risk levels (e.g., High predicted as Medium, 2,067 cases) — an acceptable but non-zero risk of under-estimating moderate-to-high risk projects.	Low–Medium

5. Dependencies

- Python environment with pandas, numpy, scikit-learn, xgboost, and joblib installed
- Access to Google Colab or a local Jupyter environment for training and evaluation
- Availability of the source dataset file (project_risk_dataset.csv)
- Mentor review and sign-off at defined checkpoints (Infosys Springboard AI Internship)
- For the pending deployment phase: a hosting environment supporting a standard Python web stack

6. Delays and Scope Changes

No formal schedule slippage has been recorded; however, the project underwent three dataset pivots before settling on the final dataset, which represents the primary source of rework during the project. Each pivot was driven by a rigorous, documented accuracy comparison rather than an ad hoc change, and is captured in the Key Decisions Log below.

7. Key Decisions Log

#	Decision
1	Started with a manually collected 4,000-row dataset; used Gaussian Copula (SDV) synthetic augmentation to reach 10,500 training rows since real data alone was insufficient (mentor-approved approach).
2	After finding a modeling ceiling (~55% at 4-class), regrouped the target into a 3-class framing (merging Critical+High into "Elevated") and switched from Random Forest to XGBoost, improving accuracy to 74%.
3	Sourced a much larger, cleaner 200,000-row dataset (project_risk_dataset.csv), eliminating the need for synthetic augmentation entirely.
4	Discovered data leakage: several raw columns described project outcomes rather than pre-project plans; including them produced a misleading 99.7% accuracy.

5	Excluded all 11 outcome/leakage columns, retrained on the remaining 19 pre-project features, and validated a genuine, defensible 80.4% accuracy on held-out real data.
6	Evaluated two further candidate 200K+ datasets against the same leakage-free standard; both scored lower (59.6% and 47.5%) and were rejected in favor of the original 200K dataset.
7	Ran an ablation test on the region feature; confirmed negligible impact (+0.09pp) and retained it since it does no harm.
8	Selected XGBoost as the final model after a controlled comparison against Random Forest and Logistic Regression.
9	Built and delivered a 21-slide presentation and a 16-page detailed project report documenting the full storyline, including the rejected alternatives and the leakage correction.
10	Completed a separate mentor-assigned EDA/data-cleaning/Risk-Flag exercise using a different dataset copy appropriate for that descriptive-analytics task, since the forecasting dataset lacked the specific columns (progress, pending tasks, bugs) that exercise required.

8. Outcome Summary

Net result to date: a validated, leakage-free AI risk forecasting model achieving 80.4% held-out accuracy (vs. 45% baseline), fully documented across a presentation, a detailed report, and this six-document knowledge base. The primary remaining milestone is deploying the model behind a working web application.

9. Planned Inference Code (Pending Work)

The following is the planned shape of the inference function the backend API will use once built, loading the already-trained and saved model artifacts:

```
import joblib
import pandas as pd

model = joblib.load("final_risk_model.joblib")
encoders = joblib.load("feature_encoders.joblib")
label_encoder = joblib.load("label_encoder.joblib")

def predict_risk(project_details: dict):
    df = pd.DataFrame([project_details])
    for col, enc in encoders.items():
        df[col] = enc.transform(df[col].astype(str))

    pred_idx = model.predict(df)[0]
    proba = model.predict_proba(df)[0]

    return {
        "predicted_risk": label_encoder.inverse_transform([pred_idx])[0],
        "confidence": round(float(proba[pred_idx]) * 100, 1),
        "probabilities": dict(zip(label_encoder.classes_, proba.round(3))),
    }

# Example call (pending web form wiring):
# predict_risk({"project_type": "Software Development", "priority": "High", ...})
```

This function is not yet wired to a live web server — that integration is the single largest remaining item in the project's pending task list (Section 3).

10. Frequently Asked Questions

Q: Is the project finished?

The modeling and analysis work is finished and validated. The interactive web application is designed but not yet deployed — see Section 3, Pending Tasks.

Q: What is the single biggest risk to this project right now?

The Critical-class recall gap (67.6%), because it directly affects the tool's core purpose of catching high-risk projects early.

Q: How many times did the dataset choice change during the project?

Three pivots: from the original 4,000-row dataset, to the final 200,000-row dataset, with two other 200K+ candidates tested and rejected along the way. All are documented in the Key Decisions Log.

Q: What would need to happen for this to become a deployable product?

Building the backend API (see the inference code above), connecting it to a working front-end form, hosting both, and ideally improving Critical-class recall before real-world use.

Q: Who approves major decisions on this project?

The Infosys Springboard AI Internship mentor reviews and signs off at defined checkpoints, as noted in the Dependencies section.